

Transformers

A walkthrough of how modern language models understand text — from attention to tokenizers.

Mosharof Hossain

AI Engineer @ Brain Station 23

What is NLP? Why is it hard?

What it is

- A field of linguistics and machine learning focused on understanding everything related to human language — not just recognizing words, but grasping context.
- Common applications: sentiment analysis, spam detection, grammar checking, named entity recognition, text generation, question answering, translation, summarization, and speech recognition.

Why it's hard

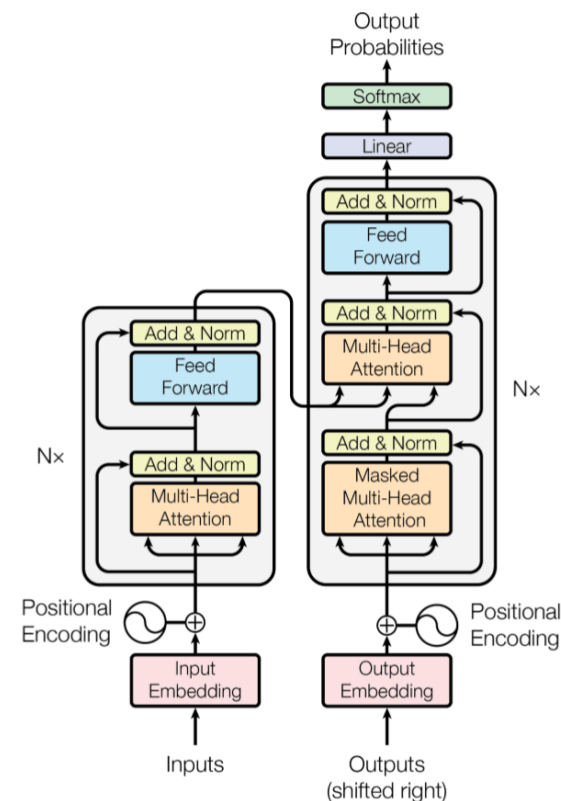
- Humans grasp meaning intuitively ("I am hungry"); machines need carefully designed text representations
- Ambiguity, cultural nuance, sarcasm, and humor need deep context
- Models learn statistical patterns, not true comprehension
- Hallucinations and bias — confident but wrong, or reproducing training-data prejudice

Transformer History Timeline

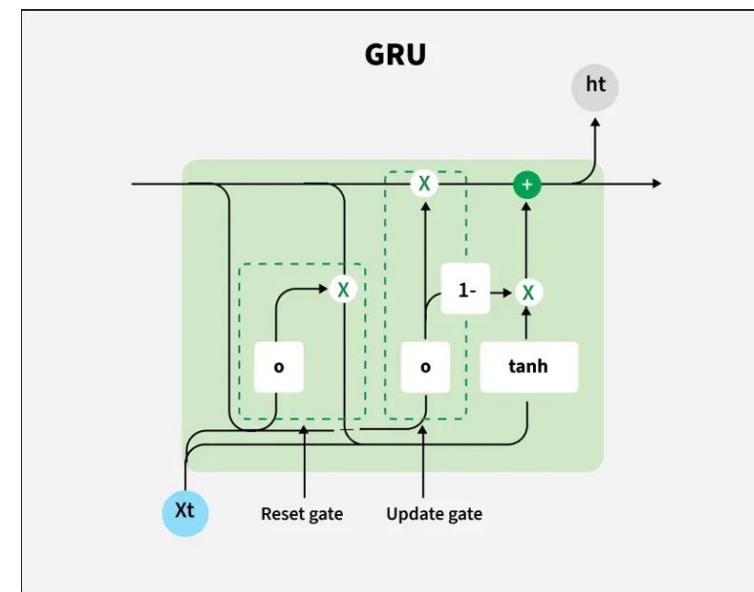
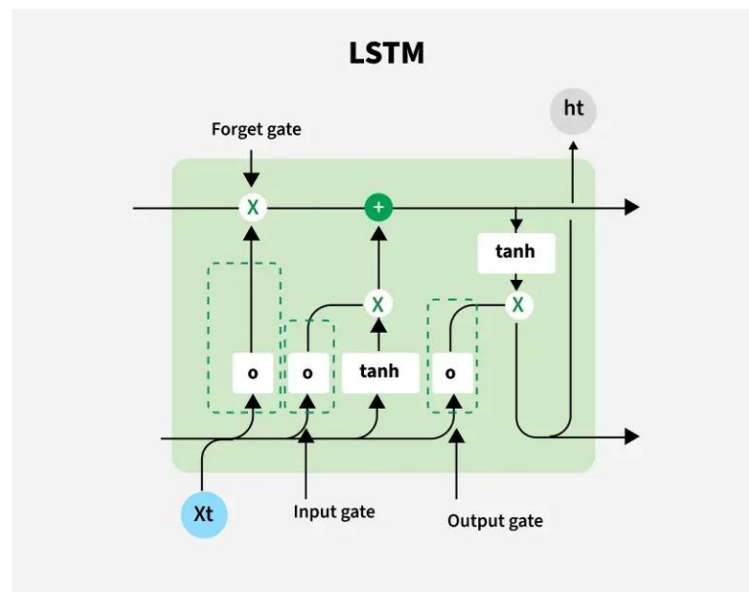
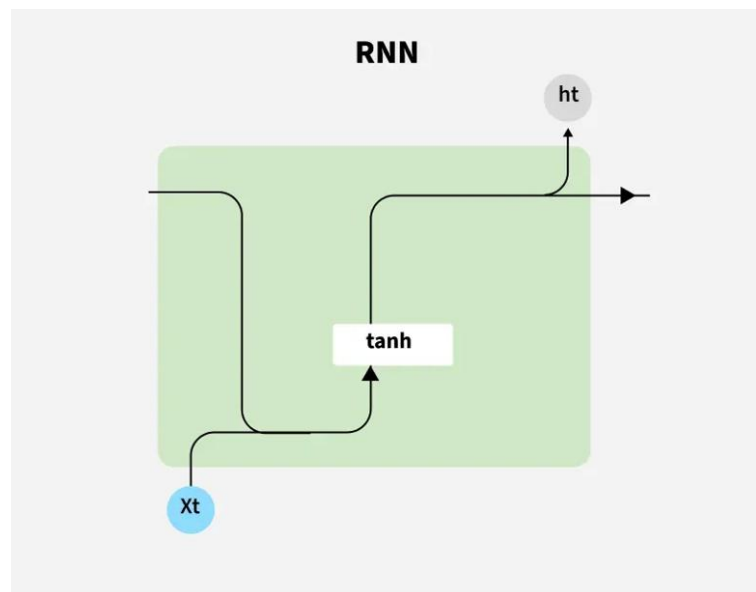
Jun 2017	Transformer architecture introduced, focused on translation
Jun 2018	GPT — first pretrained Transformer model
Oct 2018	BERT — improved sentence representation
Feb 2019	GPT-2 — larger, initially withheld for ethical reasons
Oct 2019	T5 — sequence-to-sequence architecture
May 2020	GPT-3 — zero-shot learning without fine-tuning
Jan 2022	InstructGPT — better instruction-following
Jan 2023	Llama — strong multilingual generation
Mar 2023	Mistral 7B — outperformed larger models
May 2024	Gemma 2 — lightweight open alternative
Nov 2024	SmolLM2 — strong performance at 135M–1.7B params

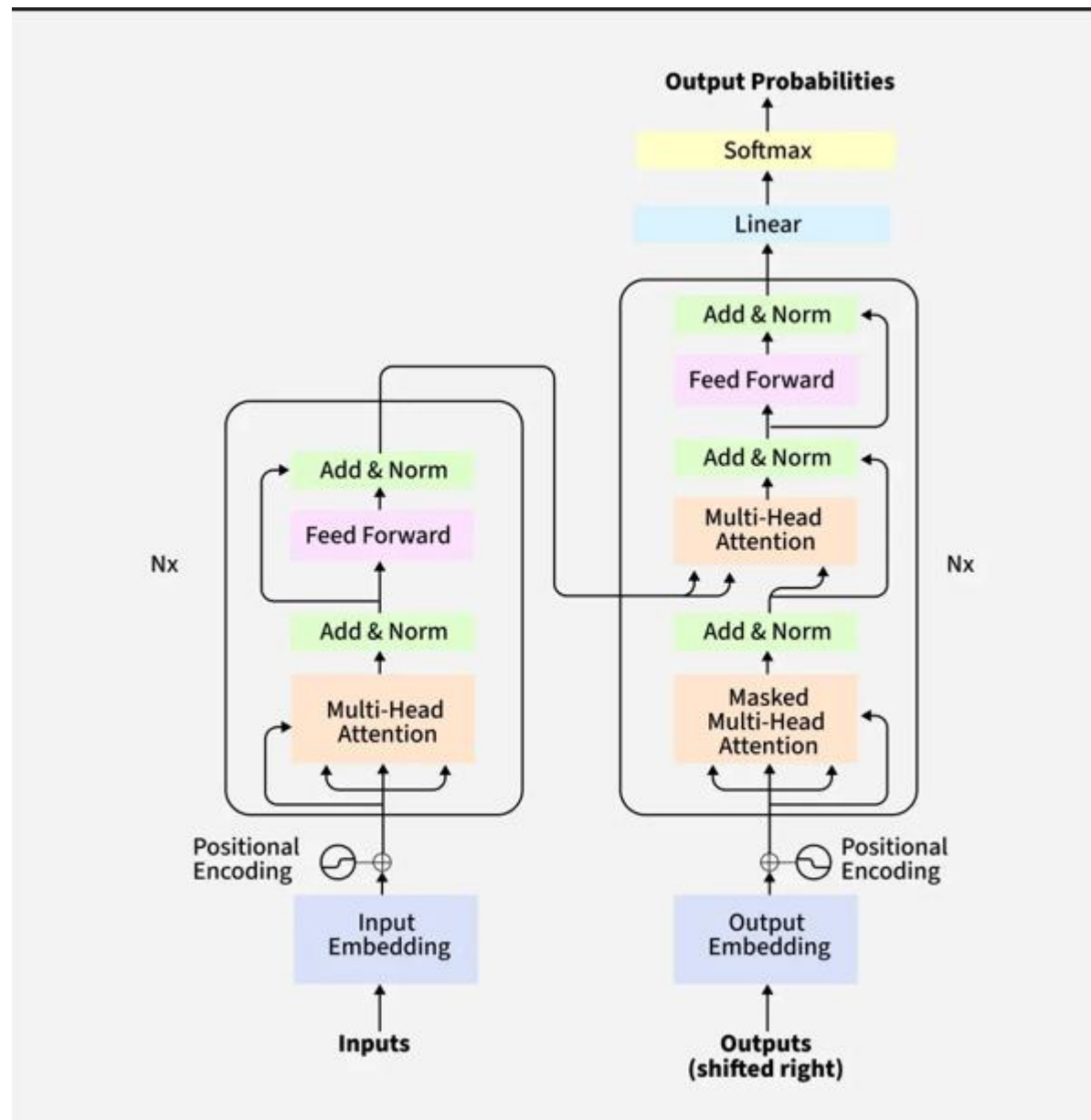
Attention Is All You Need

- The paper that named the mechanism: attention layers tell the model which words in the input to focus on — and how strongly — while processing each token.
- Every token forms a representation informed by every other token, weighted by relevance
- Removed the need for recurrence entirely — the title says it literally
- Became the foundational building block for BERT, GPT, T5, and everything after



From RNNs to Transformers — What Changed





Transformer

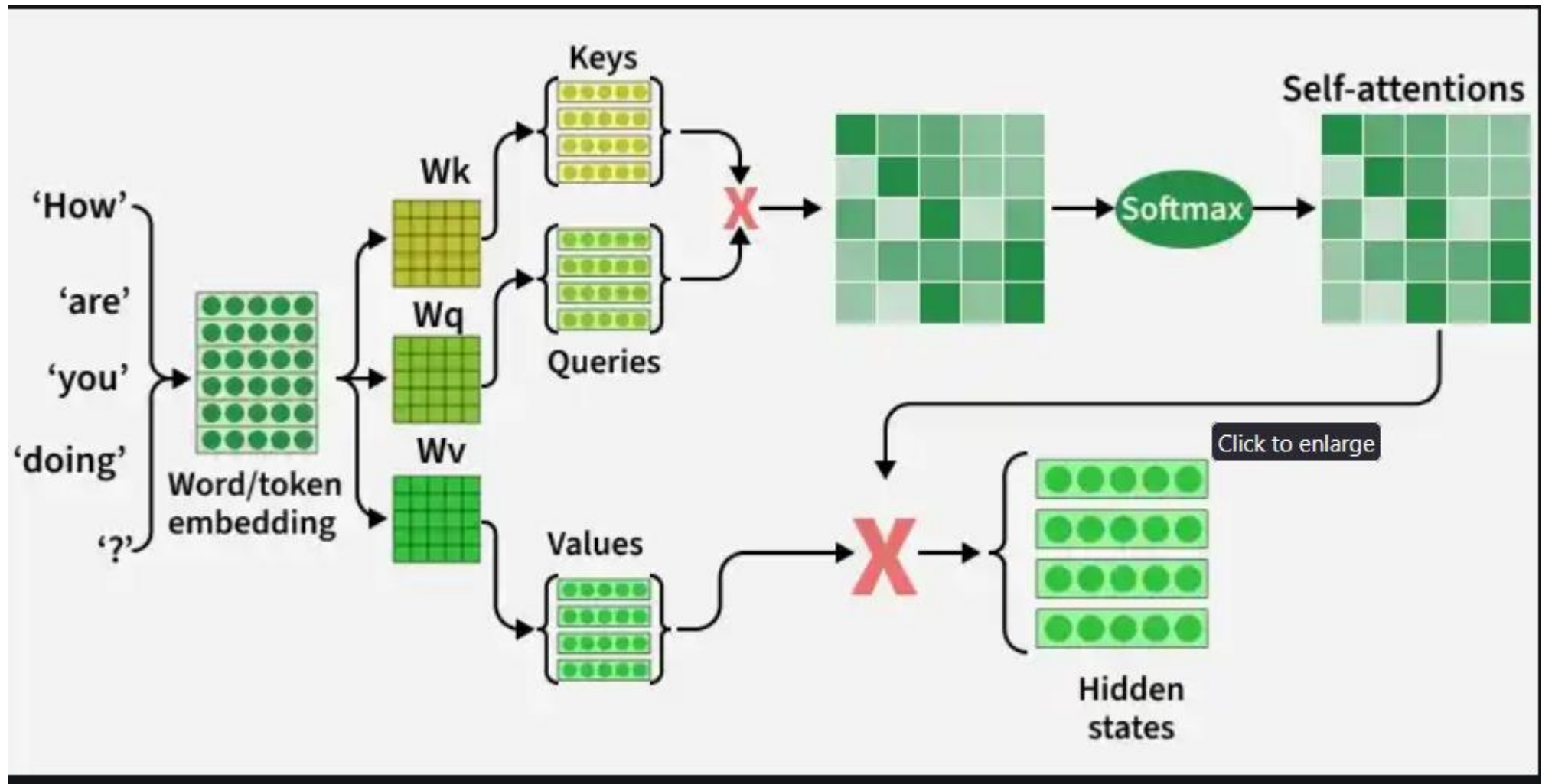
How Attention Works

Query, Key, Value

- Every token gets 3 learned vectors: Query (what am I looking for), Key (what do I contain), Value (what do I actually offer)
- $\text{Score} = \text{Query of token A} \cdot \text{Key of token B}$ — how relevant B is to A
- Scores are scaled and passed through softmax $\rightarrow$ attention weights (sum to 1)
- Output for token A = weighted sum of all Values, using those weights

Why it works

- Each token ends up as a blend of the whole sentence, not just itself
- Weights are learned — the model decides what matters per context
- Multi-head attention repeats this in parallel, each head catching a different kind of relationship (grammar, meaning, coreference)
- No recurrence needed — every token computed simultaneously

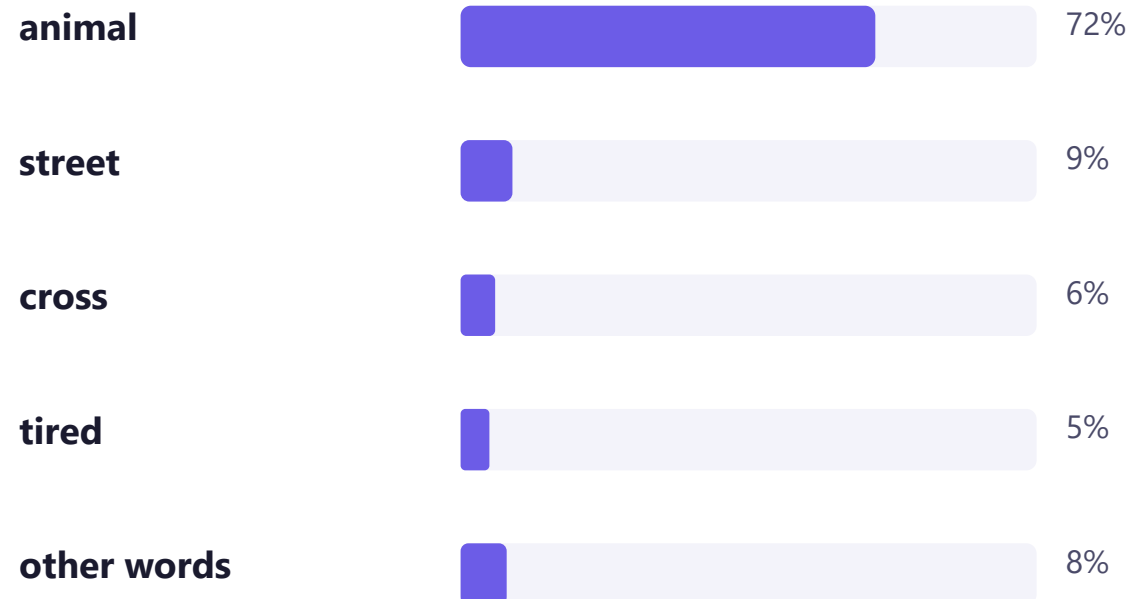


Attention Example: Resolving "it"

"The animal didn't cross the street because it was too tired."

What attention does

When the model processes the word "it", attention lets it look back at every earlier word and score how relevant each one is.



Highest weight lands on "animal" — attention resolves the pronoun by context, not word order.

Encoder–Decoder

Encoder

Processes the input and extracts features — a rich numerical representation of the text's meaning.

Decoder

Uses the encoder's representation (plus what it has generated so far) to produce the output, one token at a time.

These two components can be used together or independently — which is exactly how the three Transformer families are built.

3 Transformer Families

Encoder-only

- BERT / DistilBERT
- Bidirectional context via masked pretraining.
- Best for understanding: classification, NER.

Decoder-only

- GPT / Llama / Gemma
- Auto-regressive — predicts the next token from what came before.
- Best for generation: chat, code, reasoning.

Encoder-decoder

- BART / mBART / T5
- Bidirectional understanding + generative decoding.
- Best for transformation: summarization, translation.

Pretraining vs Fine-Tuning

Pretraining

- Starts from random weights
- Trained on massive, unlabeled text for weeks
- Self-supervised — no human labeling needed
- Learns general statistical language understanding

Fine-tuning

- Starts from a pretrained model
- Supervised learning on a small, task-specific labeled dataset
- Far less data, time, and compute than pretraining
- Transfer learning: leverages what pretraining already learned

What Can Transformers Do? (pipelines)

Text

- Sentiment analysis
- Text / zero-shot classification
- Text generation & mask filling
- Named entity recognition
- Question answering

Text (cont.)

- Summarization
- Translation
- Feature extraction

Vision & Audio

- Image classification & object detection
- Image-to-text
- Speech recognition & audio classification
- Text-to-speech

All accessible through one unified pipeline() function.

Text Generation in Detail

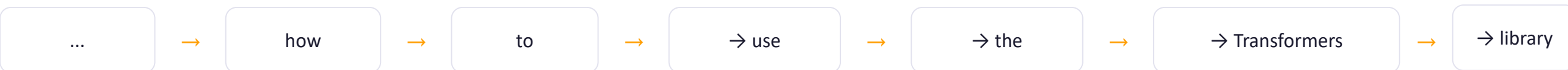
Autoregressive loop

- Model predicts one next token from the prompt so far
- That token is appended to the input
- Repeat until an end token or max length is hit

Decoding strategies

- Greedy: always pick the highest-probability token — deterministic, can repeat itself
- Sampling (temperature / top-k / top-p): adds randomness — more diverse, less predictable
- Beam search: tracks several candidate sequences at once — higher quality, costs more compute

Step-by-step: "In this course, we will teach you how to ..."



Each arrow = one forward pass through the model. Every new token becomes part of the input for predicting the next one — this is what pipeline("text-generation") runs under the hood.

Tokenizers — Text to Numbers

Models only understand numbers. A tokenizer's job: convert raw text into numerical input the model can process — using the same preprocessing scheme the model was pretrained with.

"Transformers is amazing"



tokens



input IDs



tensor

3 Tokenization Algorithms

Word-based

- Split on words, one ID each.
- Simple, but vocab explodes (500k+ words), "dog" ≠ "dogs", many [UNK] tokens.

Character-based

- Tiny vocabulary, almost no unknowns — any word builds from characters.
- But 10+ tokens per word, and characters carry little meaning alone.

Subword (BPE etc.)

- Frequent words stay whole, rare ones split into meaningful pieces — "annoyingly" → "annoying" + "ly".
- Efficient even for agglutinative languages like Turkish.

Word-Based Tokenization

How it works

- Split text on spaces/punctuation — each distinct word is its own vocabulary entry
- Vocabulary size grows huge: 500,000+ words for English alone
- "dog" and "dogs" get unrelated IDs — no shared structure
- Any word missing from the vocabulary becomes [UNK] — information lost

Example

- Sentence: "This tokenizer is annoyingly slow."
-
- annoyingly → ["annoyingly"]
-
- One token — but only if "annoyingly" is already in the vocabulary. If not: → [UNK]

Character-Based Tokenization

How it works

- Split text into individual characters — tiny vocabulary (~100 symbols)
- Almost no [UNK] — any word can be built from its characters
- Sequences get much longer: 10+ tokens per average word
- A single character carries far less meaning than a whole word

Example

- Sentence: "This tokenizer is annoyingly slow."
-
- annoyingly → [a, n, n, o, y, i, n, g, l, y]
-
- 10 tokens for one word — nothing is unknown, but the model must learn meaning from scratch, character by character.

Subword Tokenization (BPE etc.)

How it works

- Learned from data: frequent words stay whole, rare words split into meaningful pieces
- Byte-Pair Encoding (BPE) merges the most frequent adjacent symbol pairs step by step until a target vocabulary size is reached
- Balances vocabulary size against sequence length — the practical middle ground
- Handles agglutinative languages (e.g. Turkish) and unseen words gracefully

Example

- Sentence: "This tokenizer is annoyingly slow."
-
- annoyingly → ["annoying", "ly"]
-
- 2 tokens, both meaningful — "annoying" is common enough to keep whole, "ly" is a reusable suffix seen across thousands of other words.

Full Tokenization Pipeline

raw text



tokenize



input IDs



+ [CLS]/[SEP]



attention mask



decode

Encoding

- Special tokens [CLS]/[SEP] added automatically to match pretraining
- Padding: "longest" or "max_length" aligns batch sequences
- Truncation shortens sequences over the model's limit

```
tokenizer(text,  
padding=True,  
truncation=True,  
return_tensors="pt")  
# -> input_ids, attention_mask
```

Putting It Together: Text → Prediction



1. Preprocess

Tokenizer splits text into tokens and maps to input IDs, matching pretraining.

2. Model

`AutoModelForSequenceClassification` adds a task head on top of the base Transformer's hidden states.

3. Postprocess

Raw logits → SoftMax → probabilities → labeled via the model config's `id2label`.